



Développer avec **Flutter**

Le framework cross-platform by Google



Hello, moi c'est **Florian**



Développeur web/mobile à OD&B

Développeur et photographe freelance

Formation en développement Full-Stack





Et voici **OD&B**



Table des matières

01

Pourquoi Flutter ?

A quel besoin répond Flutter et dans quel but a-t-il été créé ?

02

C'est quoi ?

Comment Flutter fonctionne-t-il, est-il mieux que les autres frameworks ?

03

Comment l'utiliser ?

De quelle façon lancer un nouveau projet, quelle est son architecture ?

04

Que faut-il apprendre ?

Quelles sont les notions à apprendre et à connaître en Flutter ?



Pourquoi Flutter ?

Parce que





230 000 000 000

Nombre de téléchargements d'applications
mobiles en 2021 (d'après App Annie)

Savez-vous ce que vous **téléchargez** ?

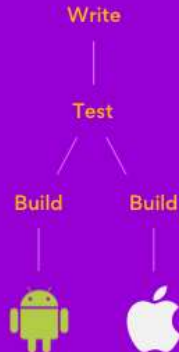


Les différences en général

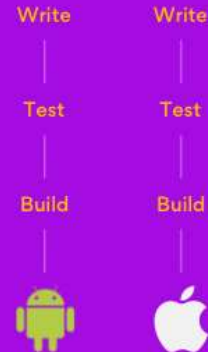
Application	Budget	Temps de développement	Performances	Expérience utilisateur
Native	Important	Important	Rapides	Très bonne
Hybride	Moyen	Moyen	Moyennes	Moyenne
PWA	Faible	Faible	Mauvaises	Mauvaise

Pourquoi de l'hybride ?

HYBRID APPS



NATIVE APPS



Dans cette liste, quelles sont les applications natives et les **applications hybrides** ?

- Google Pay
- Groupon
- Philips Hue
- Just watch
- UPS
- Facebook
- Instagram
- Uber eats
- Airbnb



Sans chercher sur Google (ce n'est pas noté 😊)

	Flutter	Xamarin	React Native	Ionic
Google pay	X			
Groupon	X			
Philips hue	X			
UPS		X		
Facebook			X	
Instagram			X	
Uber Eats			X	
Airbnb			X	
Just Watch				X



Les applications **hybrides**
sont omniprésentes
sur les stores

Et si on ne se limitait **pas juste au mobile** ?



Allez tester les exemples web : <https://flutter.github.io/samples>

Flutter répond **A un besoin**

- Un besoin de développer de plus en plus d'applications mobiles (ou autre).
- Un besoin de gagner en rapidité et un coût.
- Un besoin de ressentir du natif sous les doigts ou sous la souris.



Flutter Create

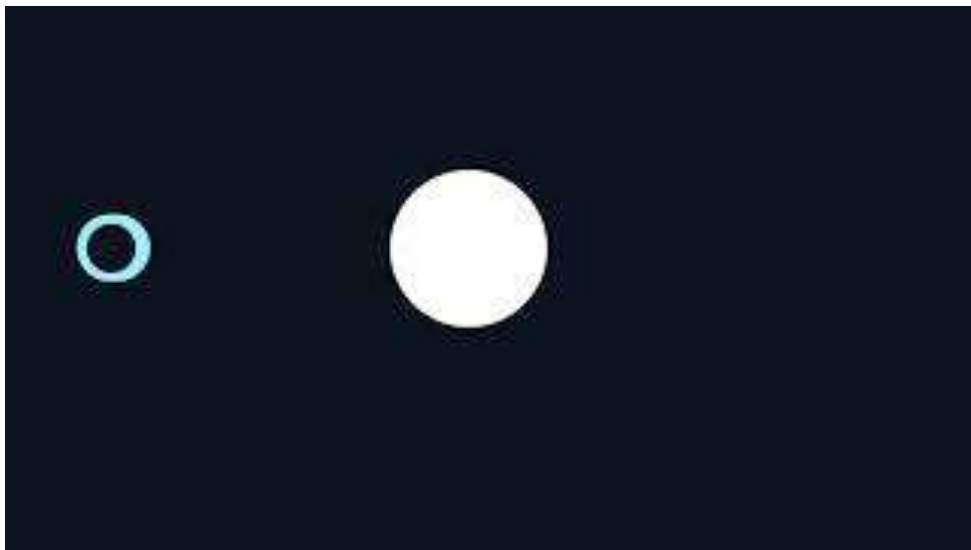
The logo features the text "Flutter Create" in a white, sans-serif font, centered over a dark blue background. The background is composed of a grid of small, colorful squares, each representing a different mobile app interface. The squares are arranged in a 4x10 grid, with the text "Flutter Create" overlaid in the center.



Il était une fois **SNCF Connect**



On essayait depuis des années de casser les silos entre les équipes iOS, Android et web



“ Sur l'Assistant SNCF, l'équipe Android avait toujours un mois d'avance sur l'équipe iOS. À l'inverse, sur OUI.sncf, l'équipe iOS était tout le temps en avance sur celle Android. À la fin, les clients n'avaient pas exactement la même application dans les mains selon leur téléphone”

“ C'est la première fois en Europe que Flutter est utilisé pour un si grand projet”

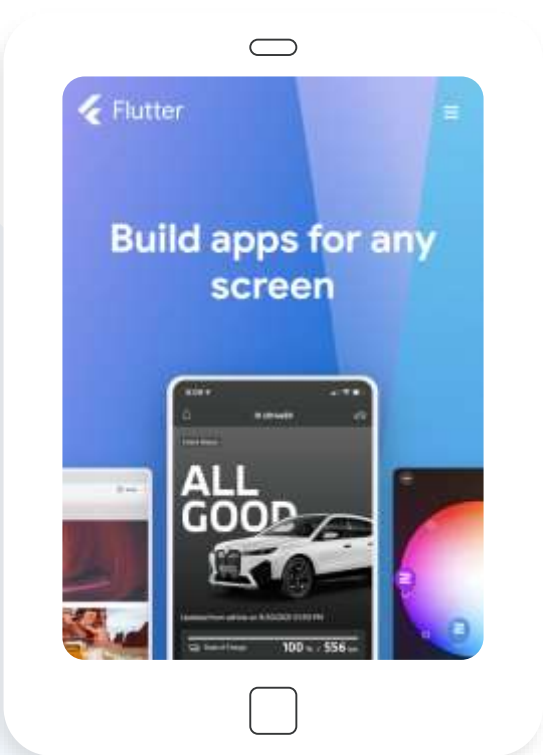
Dans la mécanique de SNCF Connect



C'est quoi **Flutter** ?

Une technologie super cool

Framework	<u>Flutter</u>	<u>Xamarin</u>	<u>React Native</u>	<u>Ionic</u>
Langage	Dart	C#	JS (React)	HTML, CSS, JS (Avec Angular ou React ou Vue)
Communauté	Moyenne mais Grandissante	Bonne	Immense	Très bonne
Performances	Excellentes	Très bonnes	Très bonnes	Moyennes
Rendu	“Canvas” (Skia et bientôt Impeller)	Composants “natifs”	Composants “natifs”	Webview



Le rendu avec Flutter

- Dessine le rendu de l'application via Skia (bientôt Impeller)
- Si des éléments changent, ils sont redessinés
- 60fps sur du 60Hz avec une image rendue toutes les 16ms (et 120fps sur du 120 Hz).
- Un rechargement à chaud



Introducing Flutter





Le langage **Dart**

“Un langage optimisé pour l’UI d’applications rapides sur n'importe quelle plate-forme”



**Créé pour
combler les
lacunes du JS**

**Proche du TS et
du Java**



Typé

Fournir **un type** a chaque variable

Null safety

Contrôler la **valeur null** dans une variable

Orienté objet

Création **d’objets** avec des **classes**, polymorphisme...



Connaissez-vous
TS/Java ?

100 *SECONDS OF*

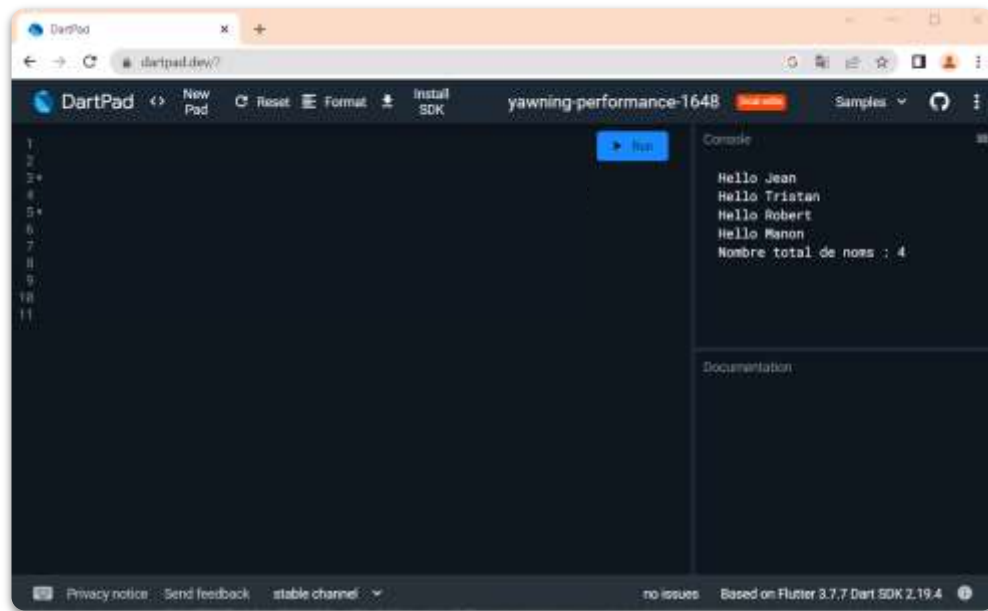


Dart

Allons **coder** sur DartPad !

Essayez d'arriver à l'affichage de droite

- En utilisant une liste de nom
- En utilisant une petite boucle
- En utilisant une méthode de la liste (pour le total de noms)



Une multitude de **plugins et packages**



Comparable à npm pour JS, Flutter dispose d'une multitude de packages disponibles sur pub.dev. Il existe 3 types de librairies :



- Des packages développés exclusivement en Dart (Http, Bloc, Crypto...)
- Des packages développés exclusivement en Flutter
- Des plugins développés avec une partie native et avec un pont vers Flutter



Cette diversité de packages permet de ne pas re-développer certaines fonctionnalités et de faciliter l'application de certains concepts.

Utilisez des plugins et des packages !



Allez sur : <https://pub.dev/> **et modifiez** votre pubspec.yaml



Attention aux spécificités de chaque plateforme

Liste de **quelques plugins et packages** qui pourraient vous donner **des idées**

1	<u>Image picker</u>
2	<u>Shared preferences</u>
3	<u>Geolocator</u>
4	<u>Lottie</u>

5	<u>Fl chart</u>
6	<u>Share plus</u>
7	<u>Camera</u>
8	<u>Flame</u>



Comment l'utiliser ?

Comme ça





Sur quel OS installer Flutter ?

Rendez-vous sur <https://docs.flutter.dev/get-started/install>



Windows

Téléchargez ou clonez le repo,
mettez à jour vos **variables PATH**.



MacOS

Faites **comme pour windows** ou
passez par **homebrew**.



Linux

Faites **comme pour windows** ou
utilisez **snap**.



ChromeOS

Téléchargez des **outils de développement**
puis faites **comme sur windows**.

Ensuite lancez la commande > **flutter doctor**



Gérer **plusieurs versions** de Flutter

A la façon de NVM pour Node, vous pourrez retrouver [FVM pour Flutter](#)



Possibilité de gérer et de mettre en cache plusieurs versions du SDK Flutter.



Possibilité de gérer une version de Flutter spécifique pour chaque projet

Les IDE pour Flutter



Android Studio

Assez **lourd** mais **complet**
et compilation android par
la même occasion



Visual Studio Code

Très **léger** et bonne prise en
charge de Flutter grâce aux
plugins VS Code



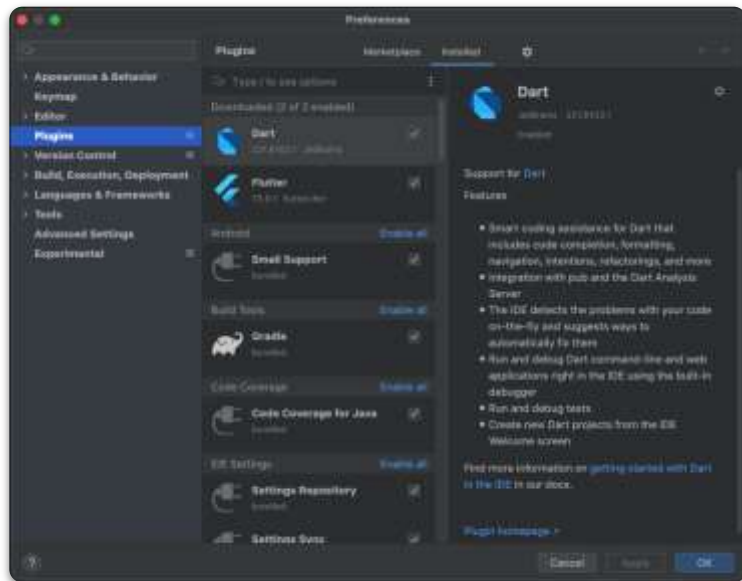
IntelliJ IDEA

IDE très **professionnel** et
complet mais **payant** !

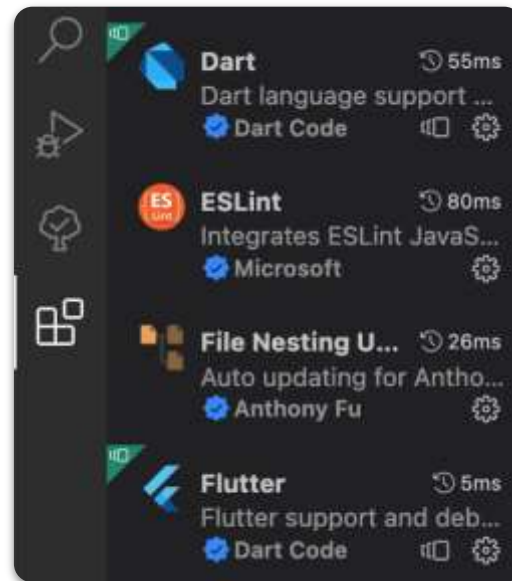


Quel est votre
IDE préféré?

Installez les **extensions** pour vos IDE



Pour **Android Studio** et **IntelliJ**



Pour **VSCode**

Développer pour **chaque plateforme**

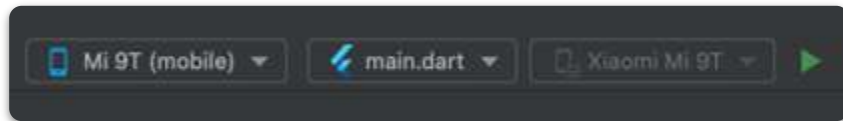
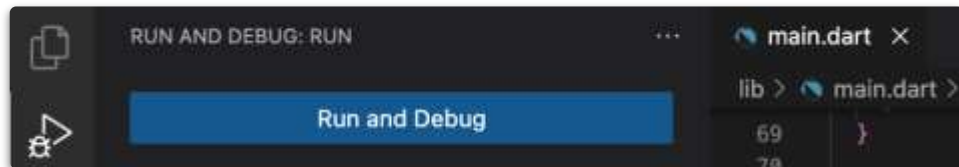


Le code Flutter peut être
**écrit sur n'importe
quelle plateforme** mais
à des **contraintes de
compilation**

Plateforme	Contraintes de compilation
Android	Android Studio (N'importe quel support)
IOS / MacOS	XCode (et donc disposer d'un mac)
Windows	Visual Studio sur Windows
Linux	Snapcraft sur Ubuntu
Web	N'importe quel support

Créez, lancez et construisez votre projet

```
> flutter create nom_du_projet  
> cd nom_du_projet  
> flutter run
```



```
> flutter build apk
```

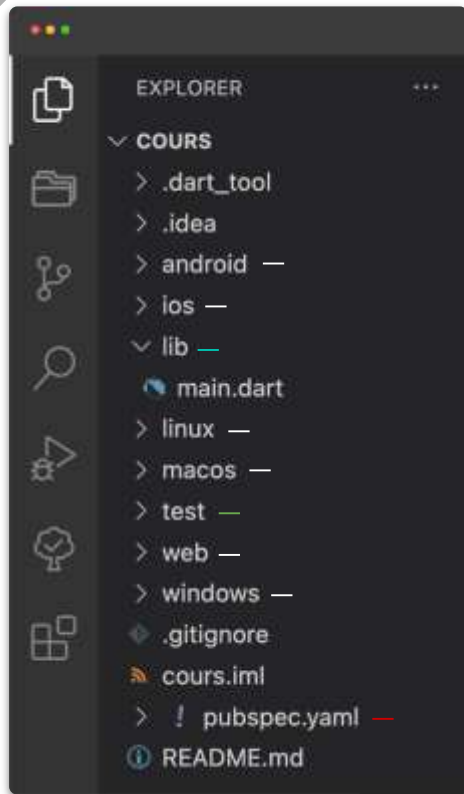
Pas de téléphone ? Passez sur **émulateur** !



Petit conseil

Privilégiez l'émulateur
iphone sur MacOS pour de
meilleures performances !

L'architecture d'un projet



- ☐ Dossiers spécifiques à chaque plateforme
- ☒ Dossier pour la partie développement Flutter
- ☒ Dossier pour les tests
- ☒ Ensemble des fichiers (nesting pour ma part) de configuration du projet et de gestion des packages



Que faut-il **apprendre ?**

Beaucoup de choses



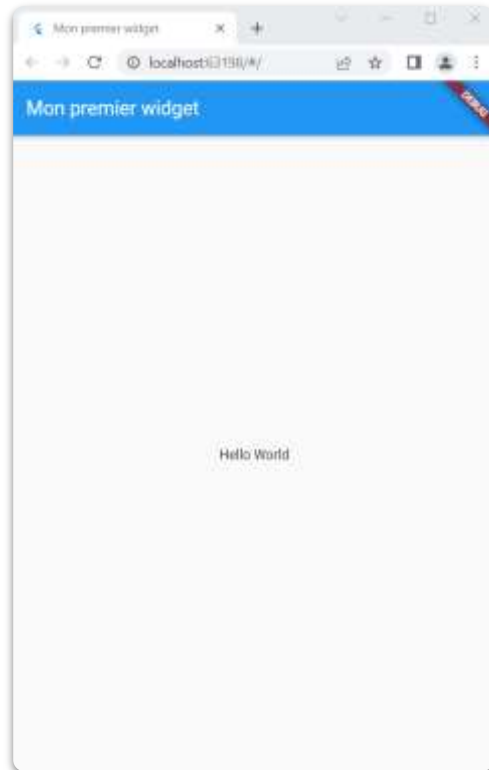
Tout est **widget** !



Tout est widget !

```
void main() {  
  runApp(const MonApplication());  
}  
  
class MonApplication extends StatelessWidget {  
  const MonApplication({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Mon premier widget',  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('Mon premier widget'),  
        ),  
        body: const Center(  
          child: Text('Hello World'),  
        ),  
      ),  
    );  
  }  
}
```

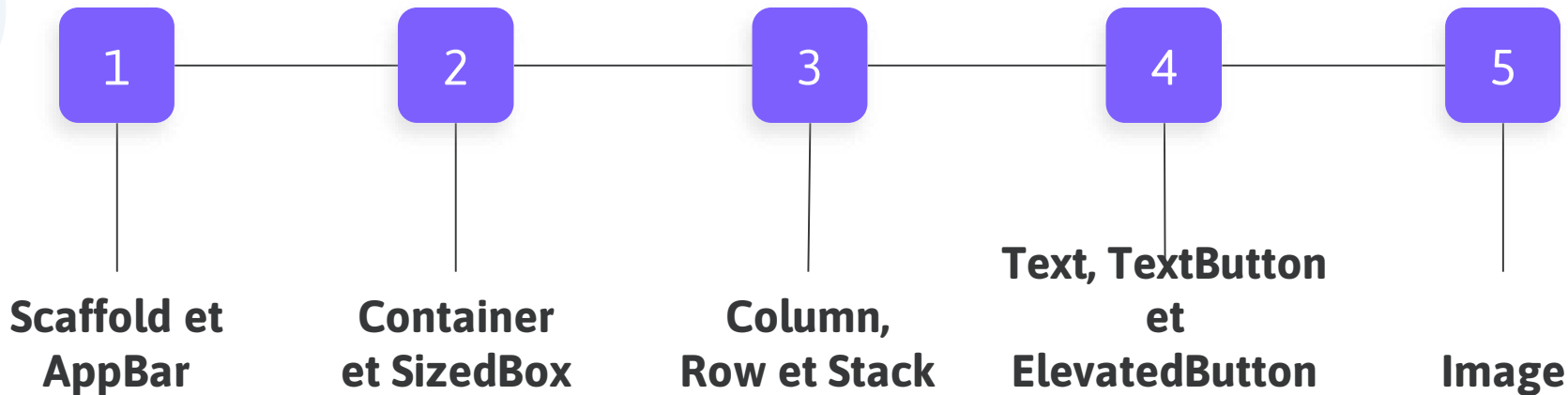
WEB



Widget of the Week



Les **principaux** Widgets



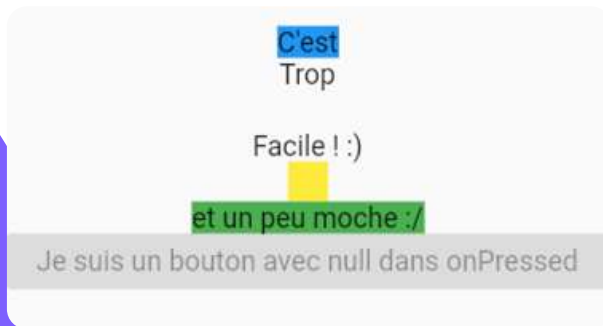
L'utilisation du **Container**





Essayez de reproduire ce rendu

Allez sur zapp.run pour **écrire du code Flutter**
directement en ligne et tenter de refaire cette vue
(Supprimez toutes les lignes après la ligne 30 pour commencer)



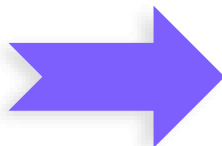
Avez-vous réussi à
vous rapprocher du
rendu attendu ?

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyPage());
}

class MyPage extends StatelessWidget {
  const MyPage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Column(
          children: [
            Container(
              color: Colors.blue,
              child: Text("C'est",
            ),
            Text('Trop'),
            SizedBox(height: 20),
            Text('Facile ! :)'),
            Container(
              width: 20,
              height: 20,
              color: Colors.yellow,
            ),
            Container(
              color: Colors.green,
              child: Text('et un peu moche :/'),
            ),
            ElevatedButton(
              child: Text('Je suis un bouton avec null dans onPressed'),
              onPressed: null,
            ),
          ],
        ),
      ),
    );
  }
}
```



C'est

Trop

Facile ! :)

et un peu moche :/

Je suis un bouton avec null dans onPressed

Les 2 grands types de Widgets

Son état ne peut plus être modifié une fois qu'il a été créé : il est immuable

Stateless

Stateful

Son état peut être modifié après sa création grâce au setState

Stateless

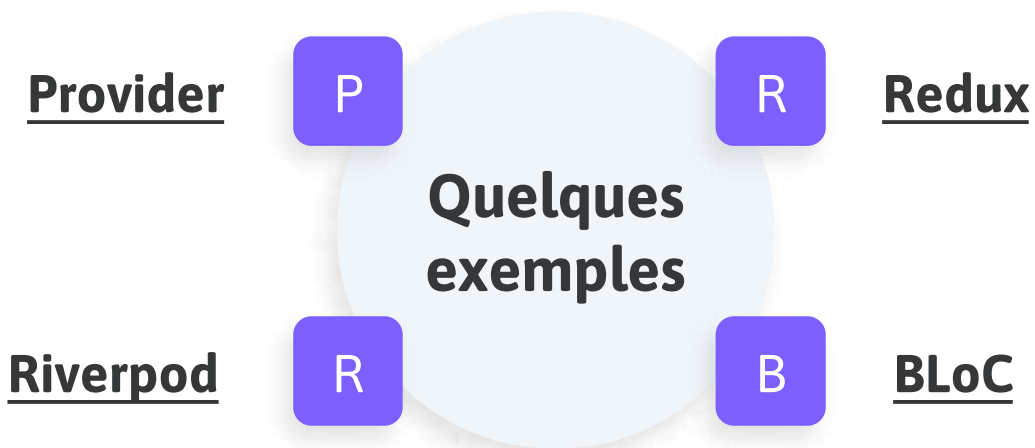
```
class MonWidgetStateless extends StatelessWidget {  
  const MonWidgetStateless({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.red,  
      child: const Text('Hello World'),  
    );  
  }  
}
```

```
class MonWidgetStateful extends StatefulWidget {  
  const MonWidgetStateful({Key? key}) : super(key: key);  
  
  @override  
  State<MonWidgetStateful> createState() => _MonWidgetStatefulState();  
}  
  
class _MonWidgetStatefulState extends State<MonWidgetStateful> {  
  int counter = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: () {  
        setState(() => counter++);  
      },  
      child: Text('Increment : $counter'),  
    );  
  }  
}
```

Stateful

La gestion d'état

Il existe un **grand nombre de packages** pour la gestion d'état en plus du `stateful` (avec son `setState`) de bas niveau proposé nativement par Flutter.

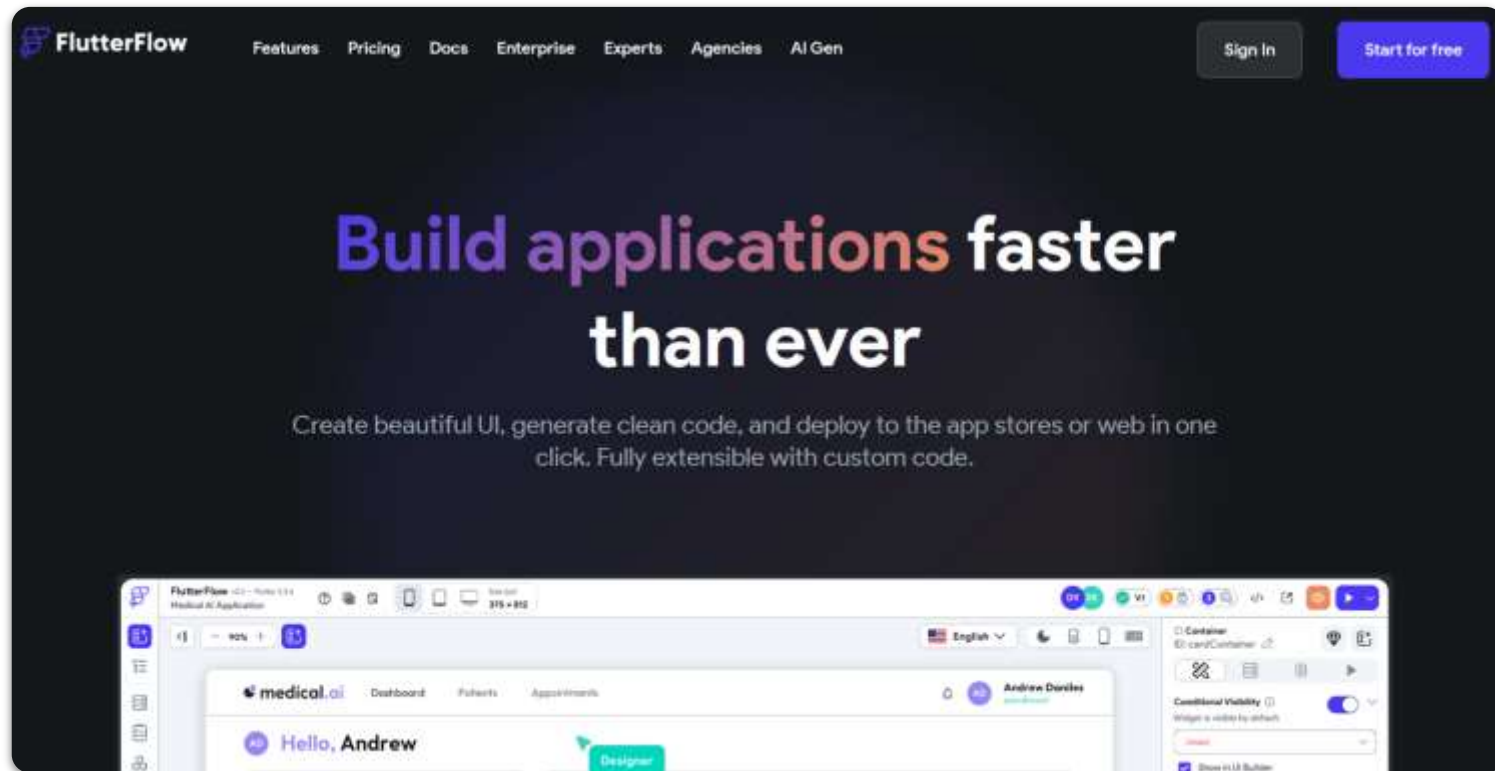


Un **récap** sur Flutter





Découvrez aussi **Flutter Flow**



Quelques **vidéos** pour apprendre Flutter

A regarder de votre côté ;)



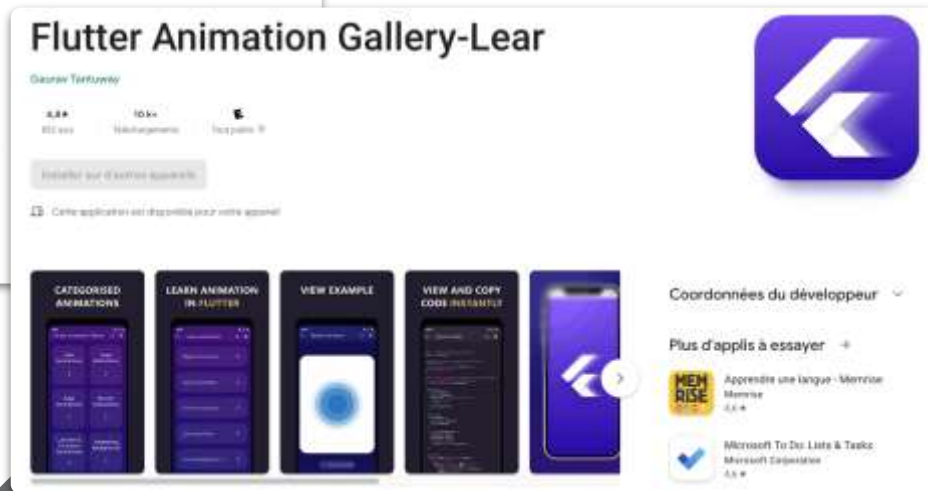
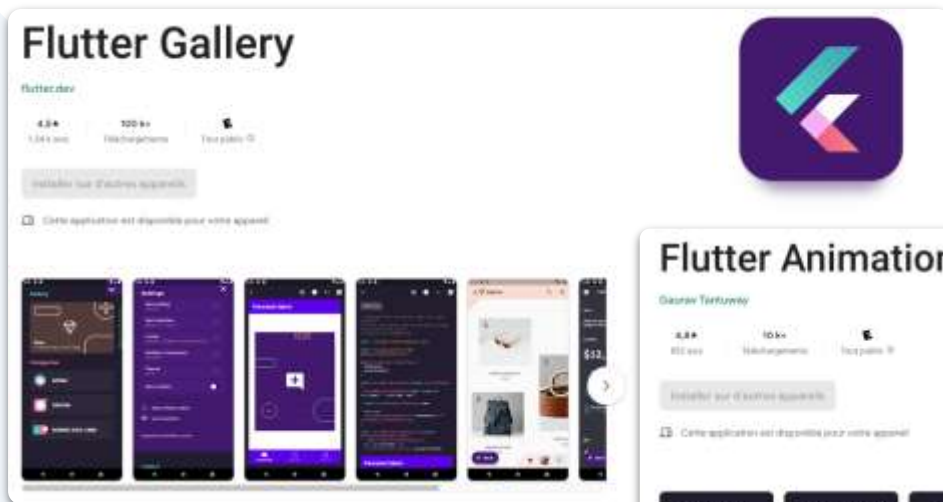
[Apprentissage Dart/Flutter \(Grafikart\)](#)



[Comprendre Flutter \(Driss AS\)](#)



Téléchargez les applications Flutter



Choisissez votre **projet**

Pour vous **exercer**, rien de mieux que de **pratiquer** !

Une application mobile en Flutter

Riche en fonctionnel

- Avec un aspect métier fort
- En utilisant du stockage (local ou firebase)
- En utilisant au moins un package de pub.dev (hors stockage)
- En utilisant une API

Équilibrée

- Avec au moins 2 contraintes de gauche
- Avec au moins 2 contraintes de droite

Riche en design

- En se basant sur un template mobile dribbble
- En utilisant des images
- En mettant en place i18n
- Avec au moins une animation



Des questions ?



Des idées pour
améliorer cette
présentation?

